Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Databases and Entity Framework

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

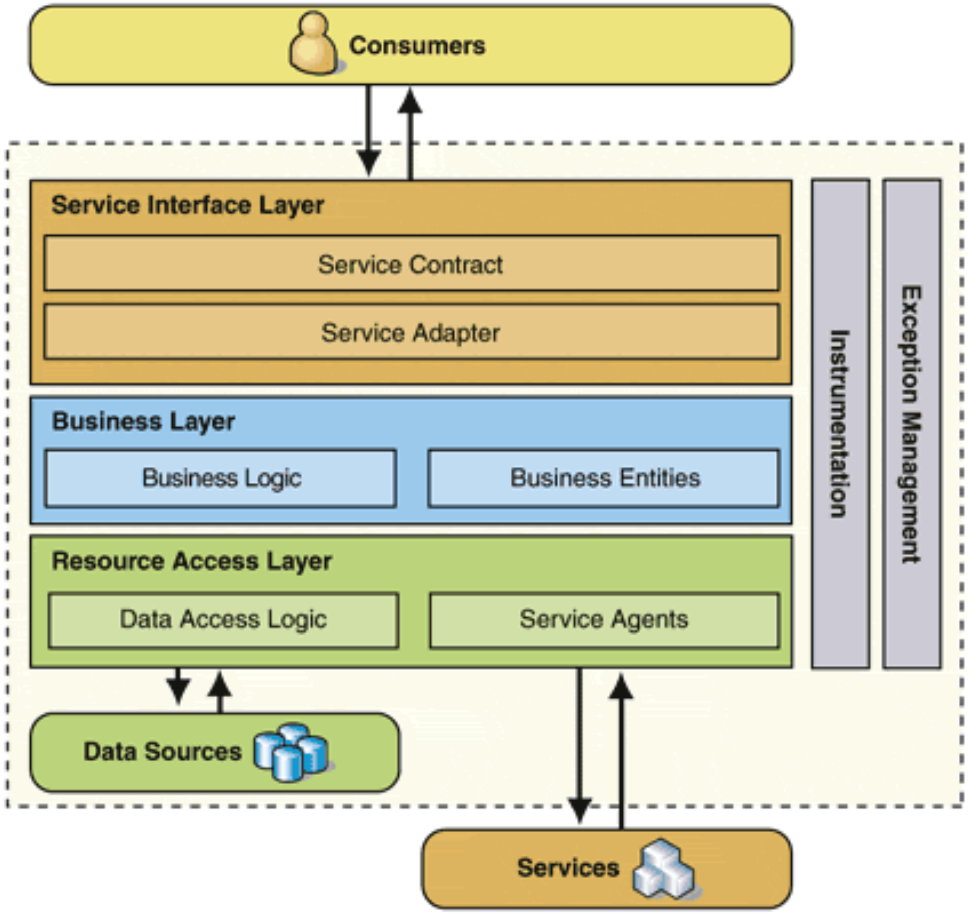
We are looking at these topics today:

- Data Storage and Databases
- Loosely Coupled vs. Strongly Coupled Data Access
- Frequently Used Database Types
- Distributed Databases
- Entity Framework

UNIVERSITY of HULL

Recap

- Note that services may have their own data sources (note the plurality)
- Services may also use another service to request resources



UNIVERSITY of HULL

Recap

- In the last session we looked at using Models in Web API to perform logical functions and access/manipulate underlying databases.
- We noted that databases don’t exist in the service layer – they exist in a separate layer called the database layer or data layer (we don’t always have to use a database)
- We also identified that it is common Microsoft practice to use Entity Framework to manage the interaction between data stored in our database and objects in our code.
- Today we will be looking at Data, Databases and the Entity Framework.

UNIVERSITY of HULL

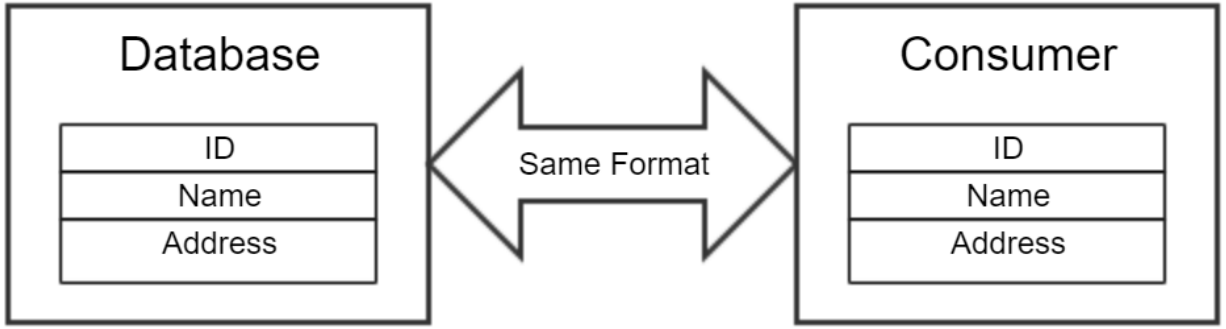
Data

- Data is what drives most of our applications.
- Resource Oriented Architectures are heavily focussed on providing access to and allowing manipulation of data.
- However, conventional wisdom (and design patterns that we have already looked at) says that we should loosely couple our application components.
  - This also extends to our data.
  - It is rarely useful to expose our data structure directly to the consumer
  - If we do, our consumer is strongly coupled to our data structure so any changes in one require a change in the other

UNIVERSITY of HULL

Strongly Coupled Data

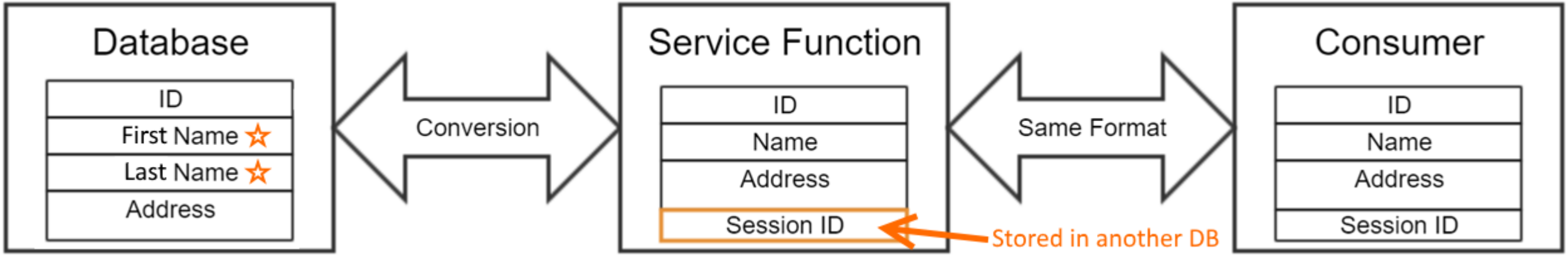
- One-to-one mapping between the data in the database and the data sent to the consumer
- Allows the consumer complete access to and manipulation of the database
- The service simply exposes the data
- Pro:
  - Very simple, makes sense with CRUD operations
  - Easy to develop for because the data structure is the same
- Con:
  - No function in the service to allow data manipulation/validation during insert/extract
  - A small change in the consumer requires a complete rewrite of the database (and vice versa)
  - Often easy to fall into the trap of including database logic in the consumer



UNIVERSITY of HULL

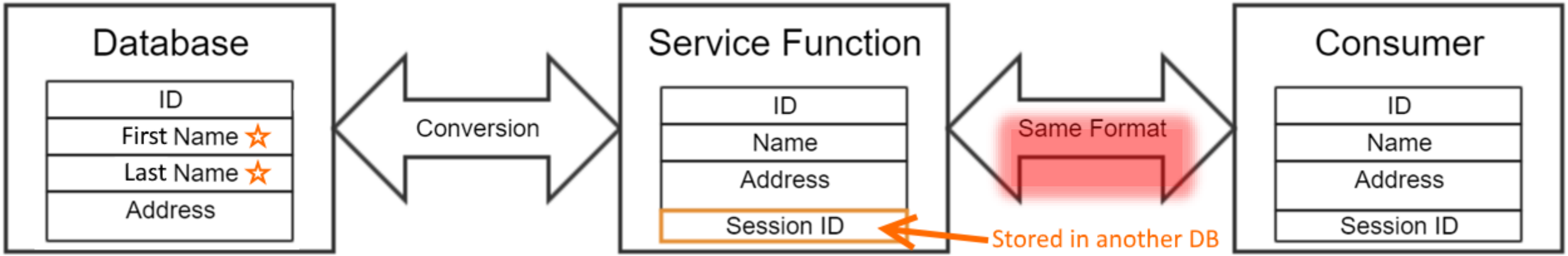
Loosely Coupled Data

- Service provides a level of abstraction between database and consumer
- Pro:
  - Data can be manipulated/validated during CRUD operations
  - Changing functionality in the consumer does not require a complete database rewrite
    - E.g. If a new field is required, rather than rewriting the table, a new table could be added and data amalgamated from both tables in the service
  - A database change doesn't result in consumers becoming non-functional (service can ensure data format is the same)
- Con:
  - Requires some translation of CRUD operations



Loosely Coupled Data

- Service provides a level of abstraction between database and consumer



- We're just talking about databases here
  - In the real world it's a good idea to introduce abstraction between the data in the Service and Consumer too!
  - This loosely couples so the Consumer isn't reliant on the data format in the Service app
  - Allowing changes on database/service/consumer that don't affect other components

UNIVERSITY of HULL

Data Conversion (Object/Database)

- Objects in memory (e.g. C# Objects) are (usually) stored in a database by translation:

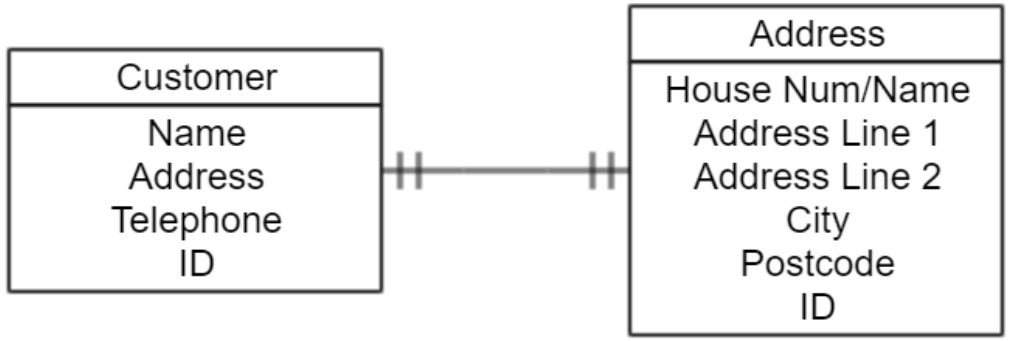
- A table is generated per object type
- Each object is given a unique ID (often an integer or GUID)
- Local value variables are converted to string, numeric or datetime types
- Local type variables are converted to relationships (and another table is created per type)
- Relationships are FOREIGN IDs, signifying which ID(s) each object is related to

Even an image *can* be stored in a database as an array of bytes or binary in string form

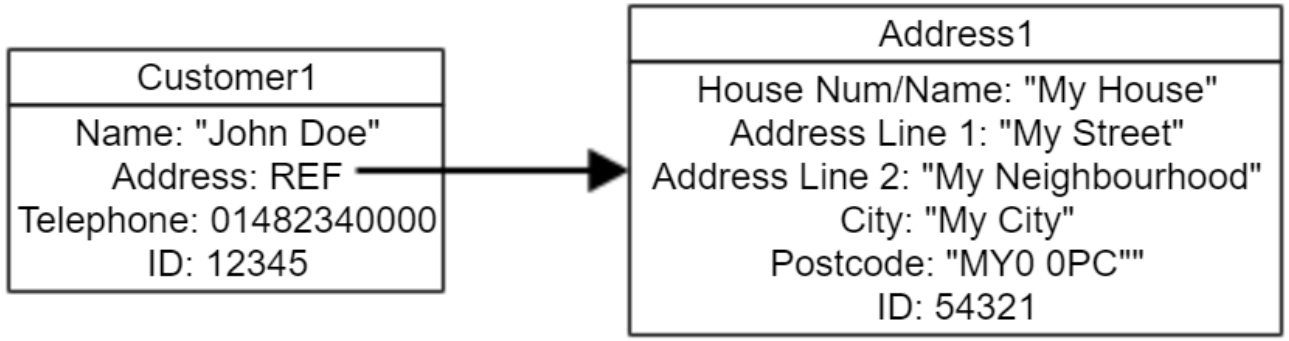
Different databases have different ways to store data (e.g. some only store strings)

Data Conversion (Object/Database)

Object Structure



Real Objects



Customer Table

Name	Address_ID	Telephone	ID
“John Doe”	**54321**	“01482340000”	12345
…	…	…	…

Address Table

House Num/Name	Address Line 1	Address Line 2	City	Postcode	ID
“My House”	“My Street”	“My Neighbourhood”	“My City”	“MY0 0PC”	**54321**
…	…	…	…		

Data Conversion - Object(s) as a Message

- Once extracted from the database, objects need to be formatted for sending via the Web

- In most cases, the only conventional way of sending object data is by sending a series of bytes which can be decoded to understand the data

- Unfortunately, this can be challenging because bytes can be encoded in different ways.

E.g. – “BOO” can be encoded into…

ASCII/Unicode as 424F4F (hex)

But 424F4F is also the hex for this RGB colour:

Or the number: 4345679

If you receive the bytes 424F4F and you don’t know what to decode it as, you don’t know what the data is!

UNIVERSITY of HULL

Data Conversion - Object(s) as a Message

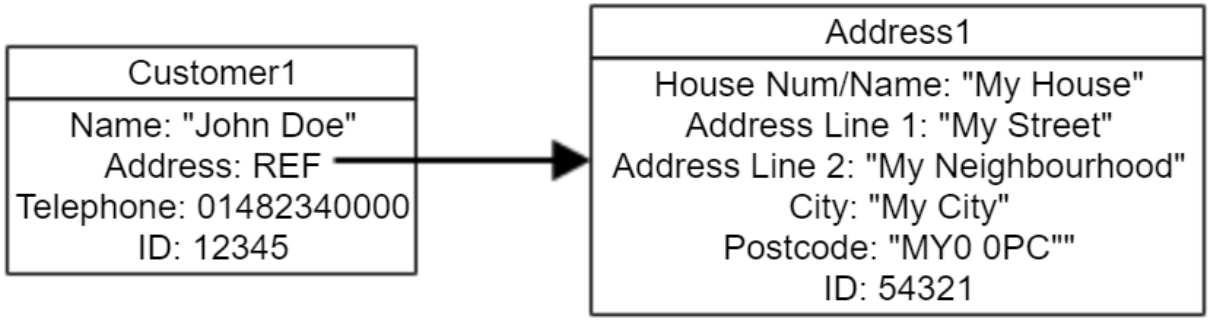
- To combat this, and allow different data types to co-exist, the whole object is often converted to a string in a way that allows it to be converted back to an object at the other end.

- The recipient is informed (usually via the header) that the bytes need to be converted to a string, and that this string will follow a format to allow recreation of the objects.

- There are two typical methods for doing this:

XML	JSON
Very human-readable	Less easy to read by a human
Easy to parse	Easy to parse
Understood by a wide variety of applications	Understood by mainly web-based applications
Quite verbose (wasteful so slower to transmit)	Less verbose than XML
Some serialisation can be difficult (e.g. relationships)	Relationships are easy to maintain via serialisation
Not precisely designed for this purpose	Designed exactly for this purpose

Data Conversion (Object/XML)

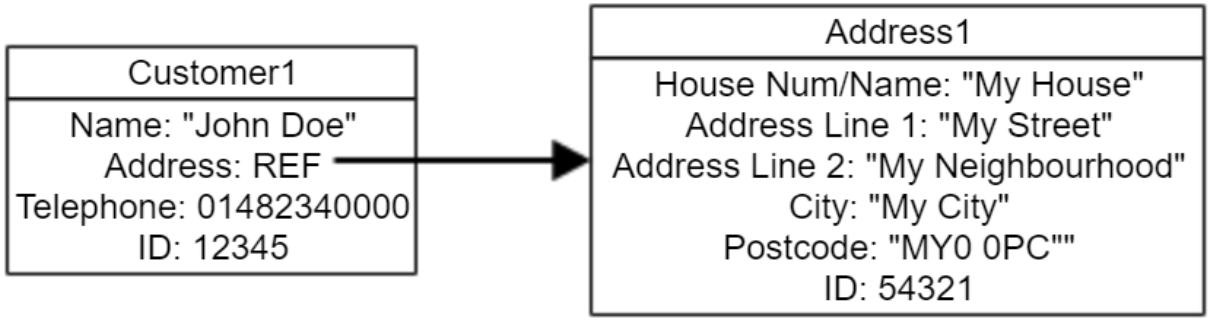



```
<Customer>
  <Name>John Doe</Name>
  <Address>
    <HouseNumName>My House</HouseNumName>
     <AddressLine1>My Street</AddressLine1>
     <AddressLine2>My Neighbourhood</AddressLine2>
     <City>My City</City>
    <Postcode>MY0 0PC</Postcode>
    <ID>54321</ID>
  </Address>
  <Telephone>01482340000</Telephone>
  <ID>12345</ID>
</Customer>
```

UNIVERSITY of HULL

Data Conversion (Object/JSON)

```
"Customer": {
  "Name": "John Doe",
  "Address": {
    "House Name/Num": "My House",
    "Address Line 1": "My Street",
    "Address Line 2": "My Neighbourhood",
    "City": "My City",
    "Postcode": "MY0 0PC",
    "ID": 54321
  },
  "Telephone": "01482340000",
  "ID": 12345
}
```



UNIVERSITY of HULL

Databases

- Services don't ***need*** to rely on any data
  - Data could all be stored in and sent from consumer
  - Perhaps the service uses no data (possibly a service that relies only on another service)
- Normally, however, a service requires some form of data
- In stateless servers, data should be stored somewhere when not in use by a current connection
- Often this data storage is done using a database because:
  - Databases are designed to make I/O operations quick
  - Common tasks can be stored as procedures in the database logic itself
  - Databases can get very, very large with little performance loss
- Storage in XML or a .txt file doesn't offer any of these benefits

SQL Databases

- SQL databases are the most common form of database in the world.
- SQL stands for Structured Query Language – it is the language used for talking to the database
  - SQL is what allows us to do our CRUD operations on the underlying database
- The underlying database is actually a Relational Database Management System (RDBMS)
- The RDMBS format is what we've been using in this lecture so far:
  - Tables which hold many rows and columns of data
  - Each column is a data field
  - Each row constitutes a separate entry
- A database schema defines the structure of tables and columns for any particular database

UNIVERSITY of HULL

NoSQL Databases

- Some developers have become aggravated with SQL databases
- They argue that:
  - The schema is limiting – it means that changes to the database require a complete schema/database rewrite
  - Relationships are limited to ONLY entries which are allowed to be related by the schema
  - SQL databases are not particularly scalable to a large number of concurrent requests
  - The database structure can lead to lots of I/O operations across multiple tables – using time and resources

- NoSQL databases are an answer to that, although NoSQL is a broad term for a database that isn't a RDMBS
  - Primarily, NoSQL databases are non-tabular, non-relational databases
  - Some people say the term NoSQL should be changed to NoREL, especially as some databases can be queried using SQL or similar query languages

Graph Databases

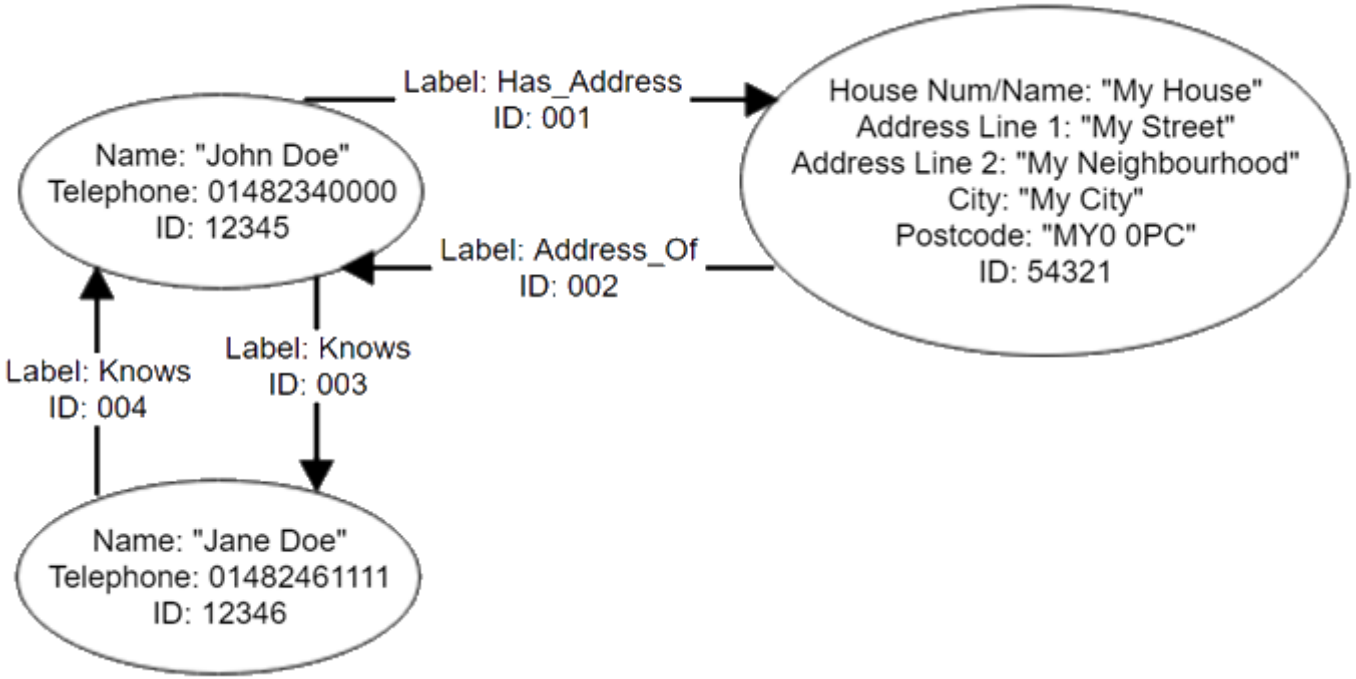
- Graph databases are a key and common type of NoSQL database

- Graphs are made from:
  - **Nodes:** An entity (e.g. Person, Address, etc.)
  - **Edges:** A one-way connecting line between nodes (e.g. Person **– has –>** Address)
  - **Properties:** Information relating to nodes or edges (e.g. Name, Address_Line_1, etc.)

- Edges are similar to relationships, so relationships DO exist in a graph database, but in a very different way…

Graph Databases

- Relationships can be made between any node… there is no schema so any node can be linked with an edge
- Often, similar nodes (e.g. ‘Person’) will have the same properties (with different values) but this is not completely necessary.
  - Consider a case where you update your person objects to include ‘Age’. In an SQL database you have to change your entire ‘Person’ schema. In NoSQL, you just start saving nodes with an ‘Age’ property. Some ‘Person’ nodes now don’t have age and some do. If you’ve loosely coupled the data, you can just modify the data as it comes out of the database to add ‘Age: Null’ where appropriate.



- Relationships can also have properties, defining what the relationship is and any other information about the link.
  - e.g. between a film and its actors, each person could have the property ‘Label: actor’ but also Type: ‘lead’, ‘support’ or ‘background’

UNIVERSITY of HULL

Graph Databases vs. SQL Databases

- Graph databases can save you lots of time:
  - Relational databases require a lot of searching through tables to follow relationships
  - Graph databases simply follow pointers to other records
  - Relationships can be made between any entries which is great if you don't know your relationships at design time.
- However, graph databases are arguably more complex, harder to implement, harder to maintain, and harder to design queries for.
- Relational databases are still perfectly good for flat or nearly-flat structures (where there are few relationships between tables)
- Graph databases are far faster when there are lots of relationships and where there is lots of searching

Distributed Databases

- A large research area over the past decade has been Distributed Databases.
- A distributed database is data stored in more than one physical device.
- This has the benefits of:
  - **Data Integrity:** Less chance of someone inserting something fraudulent if there are lots of places where someone would have to make a change
  - **Scalability:** More concurrent databases means more possible concurrent users
  - **Reliability:** More databases means if one goes offline, others can be used instead

- Often, a distributed database requires some form of replication between sites
- This is usually a complex process, requiring software to track changes and distribute these changes to other sites (which can be a very costly overhead).

- Sometimes, sites are fragmented, meaning that not all sites have the same data
  - This can lead to lower data availability and some difficulty in look-up

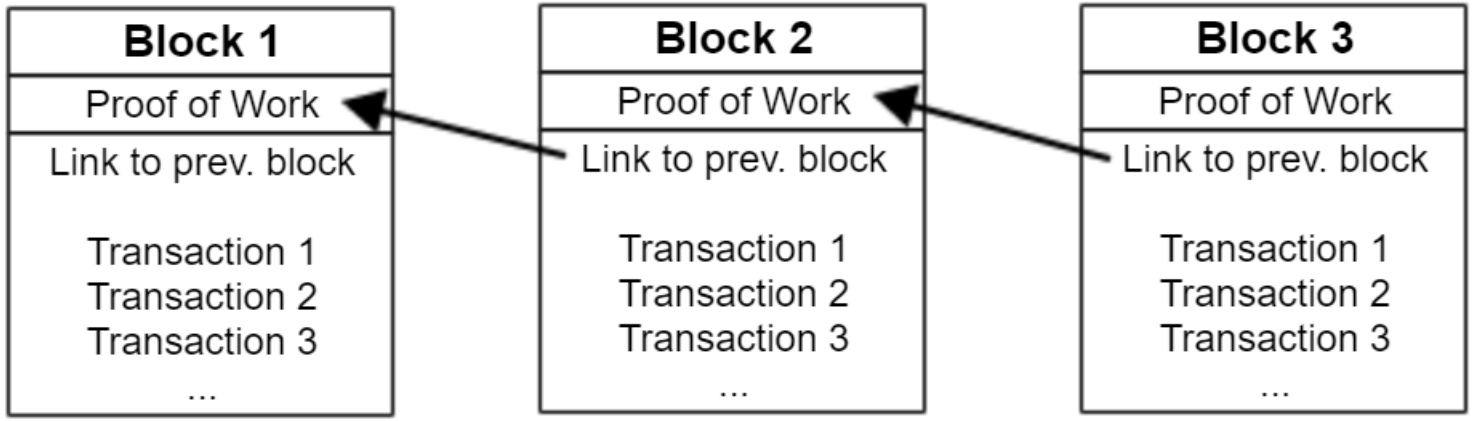
UNIVERSITY of HULL

Distributed Databases (e.g. Cryptocurrency)

- Cryptocurrency (e.g. Bitcoin) is a great example of a distributed database
- Data is stored across multiple sites and multiple devices compete to authorise transactions (through trying to determine what to add to the data to create a specific hash)
- Once the hash has been created, the data is broadcast and all of the other machines can verify that it is indeed correct
- Only upon verification can a transaction be stored into the database (in each machine)
- Each machine maintains the database so each machine stores the newly added data

UNIVERSITY of HULL

Distributed Databases (e.g. Cryptocurrency)



- Data is stored in a chain (blockchain) rather than in a more complex data structure. This is good as it makes searching very easy. The slow-down associated with the complex authorisation process ***should*** mean that only one block can be added at a time.

- Because every machine stores the same data, forging an entry would be VERY difficult because it would have to be forged in each machine.

- This is why this system is used for Cryptocurrencies!

UNIVERSITY of HULL

Entity Framework

- Entity Framework is a Microsoft framework for SQL Databases

- It takes the hassle out of converting from Objects to table entries and prevents you from having to write your own SQL queries (although you still can if you want)

- Typically, it handles the basic CRUD operations and allows you to perform queries using LINQ, as if you were searching through C# Objects.

- We have seen Entity Framework in the Labs.
  Make sure you do the lab and read around the subject because I'll only present the basics of Entity Framework Code First here.

- (You will be expected to use Entity Framework for the coursework and in labs)

UNIVERSITY of HULL

Entity Framework (Object Classes)

```
public class School
{
    public string SchoolID { get; set; }
    public string schoolName { get; set; }
    public ICollection<Student> students { get; set; }
}

public class Student
{
    public Student() {  }
    public int ID { get; set; }
    public string Name { get; set; }
    public DateTime Date_Of_Birth { get; set; }
}
```

UNIVERSITY of HULL

Entity Framework (Contexts)

```
public class SchoolContext : DbContext
{
    public SchoolContext() : base()  { }
    public DbSet<Student> Students { get; set; }
    public DbSet<School> Schools { get; set; }
}
```

Once this is done, Entity Framework can be asked to create a database!

It will make one for you using the objects you have designated in this DbContext Class

<u>If you are changing the objects you must do a **database migration** to change the schema</u>

Entity Framework (Access and saving)

```
using (var ctx = new SchoolContext())
{
    // Create
    Student std = new Student() { StudentName = "New Student" };
    ctx.Students.Add(std);

    // Read
    Student s = ctx.Students.FirstOrDefault(s => s.StudentName == "New Student")

    // Update
    s.StudentName = "Updated Student";


    // Delete
    ctx.Students.Remove(s);


    // Make the changes to the underlying database!
    ctx.SaveChanges();
}
```

UNIVERSITY of HULL

Entity Framework (Access and saving)

By default, Entity Framework uses lazy loading:

- No database access is made until the data is requested.
- This makes the system quick to run BUT is slower when data is required
- Not really noticeable unless you are accessing lots of data

You can force Eager Loading if you need to - for speedy data access

- But if you are loading lots of data this will have a big load time impact when it is loaded

Entity Framework (Access and saving)

DBContext does loading/saving but is not thread safe so…

You should create a single new instance of DBContext, then dispose of it, e.g:

```
using (var ctx = new SchoolContext())
{  …  }
```

Unless… you are using a framework like ASP.NET Core – in which case you can add Entity Framework as a Service and the framework handles the creation of a single instance for you.
In this case you should pass around the single instance you're given and not make others.

The underlying database manages multi-threaded access, race conditions, etc.
between different DBContexts – but you can configure how it does this.
-- We'll look at concurrency in a future lecture

Resources

- Entity Framework (Microsoft):
  - https://docs.microsoft.com/en-us/ef/

- Oracle Perspective on Distributed Databases:
  - https://docs.oracle.com/cd/A57673_01/DOC/server/doc/SCN73/ch21.htm

Further reading

- Book on Database Principles:
  - Principles of Distributed Database Systems (M. Tamer Özsu, Patrick Valduriez)

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- Data Storage and Databases
- Loosely Coupled vs. Strongly Coupled Data Access
- Frequently Used Database Types
- Distributed Databases
- Entity Framework Specifics

In the next session we will look at Asynchronous Programming

UNIVERSITY of HULL